

Linux Firewall and Secure Remote Access

Laboratory 4

Student Practical Laboratory Manual

Course	CORE102 TCP IP Networking and Linux Network Tools
Instructor	Aminu Idris AMCPN
Maximum score	100 marks
Gradebook category	Practical Laboratories
Estimated duration	5 to 6 hours
Platform	Kali Linux or Ubuntu Linux in an authorised laboratory environment
Version	September 2026

Student name _____Yohanna Gayus Kasa_____

Registration number _____C11/26/OCO/17006_____

Submission date _____30th September, 2026_____

Purpose and Scenario

This laboratory develops the defensive skills required to identify exposed services, enforce host firewall rules and configure SSH key authentication. Every change includes a validation and recovery step so that security controls remain observable and reversible.

You are hardening a Linux server before it joins an internal training network. The system currently exposes services without a documented policy. You must establish a baseline, permit only approved access, test the controls from a client and secure administrative access with an SSH key.

Learning Outcomes

- Identify listening TCP and UDP services and the processes that own them.
- Explain INPUT, OUTPUT and FORWARD traffic in Linux packet filtering.
- Create, verify, order and remove UFW rules safely.
- Inspect iptables or nftables compatibility rules and counters.
- Generate and deploy an Ed25519 SSH key pair.
- Validate SSH configuration before reloading the service.
- Test firewall and SSH controls from a separate authorised client.
- Document rollback steps and residual risk.

Environment and Prerequisites

- A server VM and a client VM on 192.168.56.0/24 are recommended.
- Console access to the server VM and a fresh snapshot.
- sudo permission and OpenSSH server installed on the server.
- UFW, iptables, netcat and Python 3.
- A confirmed client-to-server route before firewall changes.

Safety and Authorisation

- Do not perform firewall changes on a remote-only system without console recovery access.
- Allow and test SSH before enabling a default-deny firewall policy.
- Keep one authenticated SSH session open while testing a new SSH configuration in a second session.
- Never submit a private SSH key. Only the public key fingerprint may appear in evidence.
- Apply rules only to the authorised laboratory hosts and remove temporary services during cleanup.

Evidence Requirements

Item	Required evidence	Included
1	Baseline service and socket inventory.	[]
2	UFW policy and numbered-rule evidence before and after testing.	[]
3	Client-side allowed and denied connection tests.	[]

Item	Required evidence	Included
4	Safe iptables custom-chain exercise and cleanup.	[]
5	SSH public-key setup, fingerprint, successful login and server validation.	[]
6	Final exposure review and rollback record.	[]

Laboratory Topology

System	Example address	Purpose
Server VM	192.168.56.10	UFW, service inventory and SSH hardening
Client VM	192.168.56.20	Authorised connectivity and rule testing
Private network	192.168.56.0/24	Host-only or internal virtual network

Part 1 Baseline and Recovery Preparation

1. Take a snapshot of the server VM and confirm local console access.

```

(kali㉿kali)-[~]
$ lxc info server-vm | grep -A 5 Snapshots
lxc info client-vm | grep -A 5 Snapshots
Snapshots:
+-----+-----+-----+-----+
| NAME | TAKEN AT | EXPIRES AT | STATEFUL |
+-----+-----+-----+-----+
| baseline | 2026/09/29 06:24 EDT | | NO |
+-----+-----+-----+-----+

(kali㉿kali)-[~]
$ lxc info client-vm | grep -A 5 Snapshots

(kali㉿kali)-[~]
$ lxc info client-vm | grep -A 5 Snapshots

(kali㉿kali)-[~]
$ lxc snapshot client-vm baseline
^C

(kali㉿kali)-[~]
$ lxc info client-vm | grep -A 5 Snapshots
Snapshots:
+-----+-----+-----+-----+
| NAME | TAKEN AT | EXPIRES AT | STATEFUL |
+-----+-----+-----+-----+
| baseline | 2026/09/29 06:28 EDT | | NO |
+-----+-----+-----+-----+

(kali㉿kali)-[~]
$ 

```

2. Record interfaces, routes, listening sockets, running services and current firewall status.

```

192.168.30.1 dev eth0 proto udp scope link src 192.168.30.10 metric 100
root@server-vm:~/core102_lab4# sudo ss -tulpn
Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port Process
udp UNCONN 0 0 127.0.0.54:53 0.0.0.0:* users:((("systemd-resolve",pid=137,fd=16))
udp UNCONN 0 0 127.0.0.53%lo:53 0.0.0.0:* users:((("systemd-resolve",pid=137,fd=14))
udp UNCONN 0 0 192.168.56.10%eth0:68 0.0.0.0:* users:((("systemd-network",pid=262,fd=21))
udp UNCONN 0 0 [fe80::216:3eff:fe6b:7a30]%eth0:546 [::]:* users:((("systemd-network",pid=262,fd=22))
tcp LISTEN 0 4096 127.0.0.54:53 0.0.0.0:* users:((("systemd-resolve",pid=137,fd=17))
tcp LISTEN 0 4096 0.0.0.0:22 0.0.0.0:* users:((("sshd",pid=904,fd=3),("systemd",pid=1,fd=85))
tcp LISTEN 0 4096 127.0.0.53%lo:53 0.0.0.0:* users:((("systemd-resolve",pid=137,fd=15))
tcp LISTEN 0 4096 [::]:22 [::]:* users:((("sshd",pid=904,fd=4),("systemd",pid=1,fd=86))
root@server-vm:~/core102_lab4# sudo systemctl --type=service --state=running --no-pager
UNIT LOAD ACTIVE SUB DESCRIPTION
console-getty.service loaded active running Console Getty
cron.service loaded active running Regular background program processing daemon
dbus.service loaded active running D-Bus System Message Bus
polkit.service loaded active running Authorization Manager
rsyslog.service loaded active running System Logging Service
ssh.service loaded active running OpenBSD Secure Shell server
systemd-journald.service loaded active running Journal Service

```

```

root@server-vm:~/core102_lab4# sudo systemctl --type=service --state=running --no-pager
UNIT                                LOAD    ACTIVE SUB    DESCRIPTION
console-getty.service              loaded active running Console Getty
cron.service                        loaded active running Regular background program processing daemon
dbus.service                       loaded active running D-Bus System Message Bus
polkit.service                     loaded active running Authorization Manager
rsyslog.service                    loaded active running System Logging Service
ssh.service                        loaded active running OpenBSD Secure Shell server
systemd-journald.service            loaded active running Journal Service
systemd-logind.service              loaded active running User Login Management
systemd-networkd.service            loaded active running Network Configuration
systemd-resolved.service            loaded active running Network Name Resolution
systemd-udevd.service               loaded active running Rule-based Manager for Device Events and Files
udisks2.service                    loaded active running Disk Manager
unattended-upgrades.service          loaded active running Unattended Upgrades Shutdown

Legend: LOAD → Reflects whether the unit definition was properly loaded.
         ACTIVE → The high-level unit activation state, i.e. generalization of SUB.
         SUB → The low-level unit activation state, values depend on unit type.

13 loaded units listed.
root@server-vm:~/core102_lab4# sudo ufw status verbose

```

```

root@server-vm:~/core102_lab4# sudo ufw status verbose
sudo iptables -L -n -v --line-numbers
sudo ip6tables -L -n -v --line-numbers 2>/dev/null || true
Status: inactive
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination
root@server-vm:~/core102_lab4# lxc exec client-vm -- nc -vz 192.168.56.10 22

```

3. Confirm that the client can reach the server before security rules are changed.

```

mkdir -p ~/core102_lab4 && cd ~/core102_lab4
date
hostname
ip -br addr
ip route
sudo ss -tulpn
sudo systemctl --type=service --state=running --no-pager
sudo ufw status verbose
sudo iptables -L -n -v --line-numbers
sudo ip6tables -L -n -v --line-numbers 2>/dev/null || true

```

```
(kali@kali)-[~]
└─$ lxc exec server-vm -- bash
date
hostname
ip -br addr
ip route
root@server-vm:~# exit
exit
Wed Sep 30 03:59:47 AM EDT 2026
kali
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 10.0.2.15/24 fd17:625c:f037:2:cf:1911:49fd:1e44/64 fe80::8d21:6f8d:4afb:3d9f/64
eth1 UP
lxdbro UP 192.168.56.1/24 fd42:f8a:e51b:426f::1/64 fe80::216:3eff:fe4a:a886/64
veth670baf3c@if5 UP
vethd47bdd0f@if7 UP
default via 10.0.2.2 dev eth0 proto dhcp src 10.0.2.15 metric 100
10.0.2.0/24 dev eth0 proto kernel scope link src 10.0.2.15 metric 100
192.168.56.0/24 dev lxdbro proto kernel scope link src 192.168.56.1
(kali@kali)-[~]
└─$
```

Port and protocol	Bind address	Process	Expected purpose	Keep or remove
22/TCP	0.0.0.0 (IPv4)	sshd (pid=904, fd=4)	SSH remote administration (IPv4)	Keep
22/TCP	:: (IPv6)	sshd (pid=904, fd=4)	SSH remote administration (IPv6)	Keep
53/TCP+UDP	127.0.0.53, 127.0.0.54 (loopback)	systemd-resolve	Local DNS stub	Keep (OS base)
68/UDP	192.168.56.10%eth0	systemd-network	DHCP client (server)	Keep (OS base)

Part 2 Controlled Test Service

Start a temporary web service so firewall behaviour can be tested without installing a permanent application.

```
mkdir -p ~/core102_lab4/web
echo 'CORE102 authorised firewall test' > ~/core102_lab4/web/index.html
cd ~/core102_lab4/web
python3 -m http.server 8080 --bind 0.0.0.0
```

In a second server terminal, confirm that the process listens on TCP port 8080. From the client, test the connection before applying firewall rules.

```
# Server
sudo ss -ltnp | grep ':8080'

# Client
curl -v http://192.168.56.10:8080/
nc -vz 192.168.56.10 8080
```

```

(kali@kali)-[~]
$ newgrp lxd
(kali@kali)-[~]
$ id -nG
lxd adm dialout cdrom floppy sudo audio dip video plugdev users netdev scanner bluetooth lpadmin wireshark kaboxer vboxsf kali

(kali@kali)-[~]
$ lxc exec server-vm -- ss -ltnp | grep ':8080'
lxc exec client-vm -- curl http://192.168.56.10:8080/
lxc exec client-vm -- nc -vz 192.168.56.10 8080
LISTEN 0      5            0.0.0.0:*      0.0.0.0:8080  0.0.0.0:*      users:(("python3",pid=1814,fd=3))
CORE102 authorised firewall test
Connection to 192.168.56.10 8080 port [tcp/http-alt] succeeded!

(kali@kali)-[~]
$

```

Part 3 UFW Policy Implementation

Remote safety: If you are connected through SSH, keep the existing session open and use the VM console for recovery.

1. Reset UFW only if the instructor confirms the VM contains no required existing rules.
2. Set default deny for incoming traffic and default allow for outgoing traffic.
3. Allow SSH from the private laboratory subnet before enabling UFW.
4. Allow the temporary web service only from the client address.
5. Add an explicit deny rule for Telnet and display numbered rules.

```

sudo ufw status verbose
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow from 192.168.56.0/24 to any port 22 proto tcp comment 'CORE102 SSH'
sudo ufw allow from 192.168.56.20 to any port 8080 proto tcp comment 'CORE102 test web'
sudo ufw deny 23/tcp comment 'Block Telnet'
sudo ufw enable
sudo ufw status numbered
sudo ss -ltnp

```

```

root@server-vm:~# sudo ufw --force reset
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow from 192.168.56.0/24 to any port 22 proto tcp comment 'CORE102 SSH'
sudo ufw allow from 192.168.56.20 to any port 8080 proto tcp comment 'CORE102 test web'
sudo ufw deny 23/tcp comment 'Block Telnet'
sudo ufw enable
sudo ufw status numbered
Backing up 'user.rules' to '/etc/ufw/user.rules.20260929_125515'
Backing up 'before.rules' to '/etc/ufw/before.rules.20260929_125515'
Backing up 'after.rules' to '/etc/ufw/after.rules.20260929_125515'
Backing up 'user6.rules' to '/etc/ufw/user6.rules.20260929_125515'
Backing up 'before6.rules' to '/etc/ufw/before6.rules.20260929_125515'
Backing up 'after6.rules' to '/etc/ufw/after6.rules.20260929_125515'

Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
Rules updated
Rules updated
Rules updated
Rules updated (v6)
Firewall is active and enabled on system startup
Status: active

    To
    --
[ 1] 22/tcp
[ 2] 8080/tcp
[ 3] 23/tcp
[ 4] 23/tcp (v6)

    Action
    ---
ALLOW IN
ALLOW IN
DENY IN
DENY IN

    From
    ---
192.168.56.0/24
192.168.56.20
Anywhere
Anywhere (v6)

# CORE102 SSH
# CORE102 test web
# Block Telnet
# Block Telnet

root@server-vm:~# █

```

Part 4 Rule Verification

1. From the authorised client, verify SSH and port 8080 access.

```

(kali@kali)-[~]
$ lxc snapshot server-vm with-ufw
lxc info server-vm | grep -A 8 Snapshots
Error: Failed creating instance snapshot record "with-ufw": Add snapshot info to the database: This "instances_snapshots" entry already exists
Snapshots:
+-----+-----+-----+-----+
| NAME   | TAKEN AT | EXPIRES AT | STATEFUL |
+-----+-----+-----+-----+
| baseline | 2026/09/29 06:24 EDT | | NO |
+-----+-----+-----+-----+
| with-ufw | 2026/09/29 09:26 EDT | | NO |
+-----+-----+-----+-----+

(kali@kali)-[~]
$ █

```

Activate Wi
Go to Settings

2. If a third authorised lab VM is available, confirm that it cannot reach port 8080 when its address is not 192.168.56.20.
3. Attempt a TCP connection to port 23 and record the result.
4. Review UFW logs or rule counters and connect the observed result to the matching rule.

```

# Authorised client
nc -vz 192.168.56.10 22
nc -vz 192.168.56.10 8080
nc -vz -w 3 192.168.56.10 23
curl http://192.168.56.10:8080/

# Server
sudo ufw status numbered

```

```
sudo journalctl -k --since '10 minutes ago' | grep -i ufw | tail -30
sudo iptables -L -n -v --line-numbers
```

```

Session  Actions  Edit  View  Help

(kali㉿kali)-[~]
└─$ newgrp lxd
(kali㉿kali)-[~]
└─$ lxc exec client-vm -- nc -vz 192.168.56.10 22
Connection to 192.168.56.10 22 port [tcp/ssh] succeeded!

(kali㉿kali)-[~]
└─$ lxc exec client-vm -- curl http://192.168.56.10:8080/
lxc exec client-vm -- nc -vz 192.168.56.10 8080
CORE102 authorised firewall test
Connection to 192.168.56.10 8080 port [tcp/http-alt] succeeded!

(kali㉿kali)-[~]
└─$ curl -v --max-time 5 http://192.168.56.10:8080/
nc -vz -w 3 192.168.56.10 8080
* Trying 192.168.56.10:8080 ...
* Connection timed out after 5001 milliseconds
* closing connection #0
curl: (28) Connection timed out after 5001 milliseconds
192.168.56.10: inverse host lookup failed: Unknown host
(UNKNOWN) [192.168.56.10] 8080 (http-alt) : Connection timed out

(kali㉿kali)-[~]
└─$ lxc exec client-vm -- nc -vz -w 3 192.168.56.10 23
nc: connect to 192.168.56.10 port 23 (tcp) timed out: Operation now in progress

(kali㉿kali)-[~]
└─$
```

Test	Expected	Observed	Matching rule or explanation
SSH from client	Allowed	Connection to port 22 succeeded	Rule [1] ALLOW IN from 192.168.56.0/24 to 22/tcp
HTTP 8080 from authorised client	Allowed	curl returned the test page	Rule [2] ALLOW IN from 192.168.56.20 to 8080/tcp
HTTP 8080 from other client	Denied	Connection timed out	Not matched by rule [2] (source not .20); default deny incoming applies
Telnet 23	Denied	Connection timed out	Rule [3] DENY IN 23/tcp (and rule [4] for IPv6)

Existing Table

UFW Rule	iptables Counter	Client Test That Hit It
[1] 22/tcp ALLOW from 192.168.56.0/24	60 packets accepted	nc -vz 192.168.56.10 22 from client-vm
[2] 8080/tcp ALLOW from 192.168.56.20	120 packets accepted	curl http://192.168.56.10:8080/ from client-vm
[3] 23/tcp DENY	180 packets dropped	nc -vz -w 3 192.168.56.10 23 from client-vm

Source line: *From* `sudo iptables -L ufw-user-input -n -v --line-numbers` on the server after all client tests.

Part 5 Safe iptables Chain Exercise

This exercise creates a custom chain but does not attach it to INPUT, OUTPUT or FORWARD. It demonstrates rule structure without changing live traffic flow.

```
sudo iptables -N CORE102_LAB
sudo iptables -A CORE102_LAB -p tcp --dport 23 -j DROP
sudo iptables -A CORE102_LAB -p tcp --dport 22 -j ACCEPT
sudo iptables -L CORE102_LAB -n -v --line-numbers
sudo iptables -S CORE102_LAB
sudo iptables -F CORE102_LAB
sudo iptables -X CORE102_LAB
sudo iptables -L CORE102_LAB 2>&1 || echo "Custom chain removed"
```

```
root@server-vm: ~
Session Actions Edit View Help
num  pkts bytes target    prot opt in     out     source         destination    tcp dpt:23
1      0      0 DROP      6  --  *      *         0.0.0.0/0      0.0.0.0/0      tcp dpt:22
2      0      0 ACCEPT    6  --  *      *         0.0.0.0/0      0.0.0.0/0
-N CORE102_LAB
-A CORE102_LAB -p tcp -m tcp --dport 23 -j DROP
-A CORE102_LAB -p tcp -m tcp --dport 22 -j ACCEPT
root@server-vm:~# sudo iptables -L ufw-user-input -n -v --line-numbers
sudo iptables -N CORE102_LAB
sudo iptables -A CORE102_LAB -p tcp --dport 23 -j DROP
sudo iptables -A CORE102_LAB -p tcp --dport 22 -j ACCEPT
sudo iptables -L CORE102_LAB -n -v --line-numbers
sudo iptables -S CORE102_LAB
sudo iptables -F CORE102_LAB
sudo iptables -X CORE102_LAB
sudo iptables -L CORE102_LAB 2>61
Chain ufw-user-input (1 references)
num  pkts bytes target    prot opt in     out     source         destination    tcp dpt:22
1      1      60 ACCEPT    6  --  *      *         192.168.56.0/24 0.0.0.0/0      tcp dpt:22
2      2     120 ACCEPT    6  --  *      *         192.168.56.20   0.0.0.0/0      tcp dpt:8080
3      3     180 DROP      6  --  *      *         0.0.0.0/0       0.0.0.0/0      tcp dpt:23
iptables: Chain already exists.
Chain CORE102_LAB (0 references)
num  pkts bytes target    prot opt in     out     source         destination    tcp dpt:23
1      0      0 DROP      6  --  *      *         0.0.0.0/0      0.0.0.0/0      tcp dpt:23
2      0      0 ACCEPT    6  --  *      *         0.0.0.0/0      0.0.0.0/0      tcp dpt:22
3      0      0 DROP      6  --  *      *         0.0.0.0/0      0.0.0.0/0      tcp dpt:23
4      0      0 ACCEPT    6  --  *      *         0.0.0.0/0      0.0.0.0/0      tcp dpt:22
-N CORE102_LAB
-A CORE102_LAB -p tcp -m tcp --dport 23 -j DROP
-A CORE102_LAB -p tcp -m tcp --dport 22 -j ACCEPT
-A CORE102_LAB -p tcp -m tcp --dport 23 -j DROP
-A CORE102_LAB -p tcp -m tcp --dport 22 -j ACCEPT
iptables: Incompatible with this kernel.
root@server-vm:~#
```

1. Label the tool, action, protocol match, destination-port match and target in each rule.

2. Explain why the custom chain has no effect until another chain jumps to it.
3. Confirm that the chain is removed at the end of the exercise.

Part 6 SSH Key Authentication

1. On the client, create a dedicated Ed25519 key pair with a strong passphrase.

```
(kali㉿kali)-[~]  
$ lxc exec client-vm -- ls -la /root/.ssh/core102_ed25519.pub  
-rw-r--r-- 1 root root 104 Sep 29 13:53 /root/.ssh/core102_ed25519.pub  
  
(kali㉿kali)-[~]  
$
```

```
(kali㉿kali)-[~]  
$ lxc exec client-vm -- bash  
root@client-vm:~# mkdir -p ~/.ssh  
chmod 700 ~/.ssh  
ssh-keygen -t ed25519 -a 64 -f ~/.ssh/core102_ed25519 -C "CORE102 authorised lab"  
Generating public/private ed25519 key pair.  
Enter passphrase (empty for no passphrase):  
Enter same passphrase again:  
Your identification has been saved in /root/.ssh/core102_ed25519  
Your public key has been saved in /root/.ssh/core102_ed25519.pub  
The key fingerprint is:  
SHA256:/H/TyXNyoCfXiIZnHyUnDEQH7oIjenvvgSkAebN/9RQ CORE102 authorised lab  
The key's randomart image is:  
+--[ED25519 256]--+  
|                .+.. |  
| .              o . |  
|o o      E o      |  
| o o    . . o o    |  
| o  . S o . + o    |  
| o . = = . . =     |  
| + = . o. oo=.    |  
| +.....B B==      |  
| .+. +.*.+o      |  
+---[SHA256]-----+  
root@client-vm:~#
```

2. Display and record the public-key fingerprint. Never display or submit the private key contents.

SHA256:/H/TyXNyoCfXiIZnHyUnDEQH7oIjenvvgSkAebN/9RQ

3. Copy the public key to the server and test a new SSH session using that key.

```
(kali@kali)-[~]
$ lxc exec server-vm -- ls -la /home/student/.ssh/
lxc exec server-vm -- cat /home/student/.ssh/authorized_keys
total 12
drwx----- 2 student student 4096 Sep 29 14:02 .
drwxr-x--- 4 student student 4096 Sep 29 14:05 ..
-rw----- 1 student student 104 Sep 29 14:02 authorized_keys
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIBcAFjk9ZZ7NIb+z492pe3TgN0mhFp2be4zXYk2AXtuY CORE102 authorised lab

(kali@kali)-[~]
$
```

```
(kali@kali)-[~]
$ lxc exec client-vm -- ssh -i /root/.ssh/core102_ed25519 -o StrictHostKeyChecking=no student@192.168.56.10 hostname
Enter passphrase for key '/root/.ssh/core102_ed25519':
server-vm

(kali@kali)-[~]
$
```

4. Confirm the server sees public-key authentication in its logs.

```
(kali@kali)-[~]
$ lxc exec server-vm -- cat /home/student/.ssh/authorized_keys
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIBcAFjk9ZZ7NIb+z492pe3TgN0mhFp2be4zXYk2AXtuY CORE102 authorised lab

(kali@kali)-[~]
$ lxc exec server-vm -- id student
uid=1001(student) gid=1001(student) groups=1001(student)

(kali@kali)-[~]
$ lxc exec client-vm -- ssh -i /root/.ssh/core102_ed25519 -o StrictHostKeyChecking=no student@192.168.56.10 hostname
Warning: Permanently added '192.168.56.10' (ED25519) to the list of known hosts.
Enter passphrase for key '/root/.ssh/core102_ed25519':
server-vm

(kali@kali)-[~]
$ lxc exec server-vm -- journalctl -u ssh --since '5 minutes ago' | grep -i 'Accepted publickey'
Sep 29 14:05:50 server-vm sshd[4085]: Accepted publickey for student from 192.168.56.20 port 50532 ssh2: ED25519 SHA256:/H/TyXNyoCfXiIIZnHyUnDEQH7oIjenvygSkAebN/9RQ

(kali@kali)-[~]
$
```

```
# Client
mkdir -p ~/.ssh
chmod 700 ~/.ssh
ssh-keygen -t ed25519 -a 64 -f ~/.ssh/core102_ed25519 -C "CORE102 authorised lab"
ssh-keygen -lf ~/.ssh/core102_ed25519.pub
ssh-copy-id -i ~/.ssh/core102_ed25519.pub student@192.168.56.10
ssh -i ~/.ssh/core102_ed25519 student@192.168.56.10

# Server
sudo journalctl -u ssh --since '10 minutes ago' --no-pager | tail -40
stat -c '%a %n' ~/.ssh ~/.ssh/authorized_keys
```

Part 7 SSH Server Hardening

Gate: Do not disable password authentication until key-based login succeeds in a separate session.

1. Back up the current SSH server configuration.

```
(kali@kali)-[~]
$ lxc exec client-vm -- ls -la /root/.ssh/core102_ed25519.pub
-rw-r--r-- 1 root root 104 Sep 29 13:53 /root/.ssh/core102_ed25519.pub

(kali@kali)-[~]
$
```

2. Create a small configuration drop-in rather than rewriting the full main file.

```
SHA256:H/TyXNyOCfXiIZnHyUnDEQH7oIjenvvgSkAebN/9RQ
```

3. Validate the configuration with `sshd -t`. Do not reload if validation reports any error.

4. Keep the original session open, reload SSH and test a second key-based session.

```
# Server
sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.core102.bak
sudo nano /etc/ssh/sshd_config.d/90-core102-hardening.conf
```

Enter the following settings in the drop-in file:

```
PermitRootLogin no
PubkeyAuthentication yes
PasswordAuthentication no
KbdInteractiveAuthentication no

sudo sshd -t
sudo systemctl reload ssh
sudo systemctl --no-pager status ssh

# Client in a second terminal
ssh -i ~/.ssh/core102_ed25519 student@192.168.56.10
```

```
(kali@kali)-[~]
$ lxc exec client-vm -- ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no -o StrictHostKeyChecking=no student@192.168.56.10 'echo "should fail"' 2
>61 | tail -5
student@192.168.56.10: Permission denied (publickey).

(kali@kali)-[~]
$ lxc snapshot server-vm with-ssh-hardening
lxc info server-vm | grep -A 12 Snapshots
^C

(kali@kali)-[~]
$ lxc info server-vm | grep -A 12 Snapshots
Snapshots:
+-----+-----+-----+-----+
| NAME      | TAKEN AT      | EXPIRES AT | STATEFUL |
+-----+-----+-----+-----+
| baseline  | 2026/09/29 06:24 EDT |             | NO       |
+-----+-----+-----+-----+
| with-ufw   | 2026/09/29 09:26 EDT |             | NO       |
+-----+-----+-----+-----+
| with-ssh-hardening | 2026/09/29 10:18 EDT |             | NO       |
+-----+-----+-----+-----+

(kali@kali)-[~]
$
```

If key authentication fails, use the VM console to restore `/etc/ssh/sshd_config.core102.bak`, remove the drop-in file, run `sudo sshd -t` and reload SSH.

Part 8 Final Exposure Review and Cleanup

1. Stop the Python HTTP server and prove that port 8080 no longer listens.
2. Remove the temporary UFW port 8080 rule by its current numbered index. Re-list the rules immediately after deletion.
3. Confirm SSH remains accessible from the authorised client.

```
lxc exec client-vm -- ssh -i /root/.ssh/core102_ed25519 -o StrictHostKeyChecking=no student@192.168.56.10 hostname
# Confirm server log
lxc exec server-vm -- journalctl -u ssh --since '5 minutes ago' | grep -i 'Accepted publickey'
Enter passphrase for key '/root/.ssh/core102_ed25519':
Enter passphrase for key '/root/.ssh/core102_ed25519':
server-vm
Sep 29 15:36:48 server-vm sshd[4866]: Accepted publickey for student from 192.168.56.20 port 57094 ssh2: ED25519 SHA256:/H/TyXNyoCfXiIznHyUnDEQH7oIjenvygSkAebN/9RQ
(kali@kali)-[~]
$
```

Activate Windows
Go to Settings to activate Windows.

```
(kali@kali)-[~]
$ lxc exec server-vm -- ufw status numbered
Status: active

To
--
[ 1] 22/tcp          ALLOW IN    192.168.56.0/24      # CORE102 SSH
[ 2] 8080/tcp        ALLOW IN    192.168.56.20       # CORE102 test web
[ 3] 23/tcp          DENY IN     Anywhere            # Block Telnet
[ 4] 23/tcp (v6)     DENY IN     Anywhere (v6)       # Block Telnet

(kali@kali)-[~]
$ lxc exec server-vm -- ufw delete 2
Deleting:
allow from 192.168.56.20 to any port 8080 proto tcp comment 'CORE102 test web'
Proceed with operation (y/n)? y
Rule deleted

(kali@kali)-[~]
$ lxc exec server-vm -- ufw status numbered
Status: active

To
--
[ 1] 22/tcp          ALLOW IN    192.168.56.0/24      # CORE102 SSH
[ 2] 23/tcp          DENY IN     Anywhere            # Block Telnet
[ 3] 23/tcp (v6)     DENY IN     Anywhere (v6)       # Block Telnet

(kali@kali)-[~]
$
```

4. Compare the final socket inventory with the baseline and explain every remaining listening service.
5. Do not delete the client private key until the instructor confirms that evidence review is complete.

```
sudo ss -tulnp
sudo ufw status numbered
# Replace N with the current number of the temporary 8080 rule
sudo ufw delete N
sudo ufw status numbered
sudo ss -ltnp | grep ':8080' || echo "Temporary web service stopped"
```

```
(kali@kali)-[~]
$ lxc exec server-vm -- ufw status numbered
Status: active

      To Action From
      --
[ 1] 22/tcp ALLOW IN 192.168.56.0/24 # CORE102 SSH
[ 2] 23/tcp DENY IN Anywhere # Block Telnet
[ 3] 23/tcp (v6) DENY IN Anywhere (v6) # Block Telnet

(kali@kali)-[~]
$ lxc exec client-vm -- ssh -i /root/.ssh/core102_ed25519 -o StrictHostKeyChecking=no student@192.168.56.10 'echo "SSH still works after cleanup"'
Enter passphrase for key '/root/.ssh/core102_ed25519':
SSH still works after cleanup

(kali@kali)-[~]
$ lxc exec server-vm -- ss -tulpn
Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port Process
udp UNCONN 0 0 127.0.0.54:53 0.0.0.0:* users:(("systemd-resolve",pid=137,fd=16))
udp UNCONN 0 0 127.0.0.53:53 0.0.0.0:* users:(("systemd-resolve",pid=137,fd=14))
udp UNCONN 0 0 192.168.56.10:68 0.0.0.0:* users:(("systemd-network",pid=262,fd=11))
udp UNCONN 0 0 [fe80::216:3eff:fe6b:7a30]:546 [::]:* users:(("systemd-network",pid=262,fd=22))
tcp LISTEN 0 4096 127.0.0.54:53 0.0.0.0:* users:(("systemd-resolve",pid=137,fd=17))
tcp LISTEN 0 4096 0.0.0.0:22 0.0.0.0:* users:(("sshd",pid=904,fd=3),("systemd",pid=1,fd=53))
tcp LISTEN 0 4096 127.0.0.53:53 0.0.0.0:* users:(("systemd-resolve",pid=137,fd=15))
tcp LISTEN 0 4096 [::]:22 [::]:* users:(("sshd",pid=904,fd=4),("systemd",pid=1,fd=55))

(kali@kali)-[~]
$
```

Activate Windows
Go to Settings to activate Windows.

Assessment and Submission

Marking Guide

Assessment area	Evidence expected	Marks
Baseline and service inventory	Complete exposure record and correct process interpretation	15
UFW policy	Safe rule order, correct scope, enabling and numbered-rule evidence	20
Firewall testing	Authorised client tests linked to rules, logs or counters	15
iptables chain exercise	Correct rule anatomy, inspection and complete cleanup	10
SSH key authentication	Secure key generation, deployment, fingerprint and successful login	20
SSH hardening and recovery	Valid configuration, controlled reload, retest and documented recovery	15
Final review and report	Cleanup, residual exposure explanation and complete evidence	5

Total: 100 marks

Submission Package

Submit one PDF report and one ZIP archive. Use your registration number in both filenames.

- REGNO_CORE102_Lab4_Report.pdf
- REGNO_CORE102_Lab4_Files.zip containing only public configuration evidence and the SSH public key file

The PDF report must place each screenshot directly below the task it supports. Every screenshot must show the command and its output. Do not submit unreadable terminal fragments or screenshots that hide the command used.

Reflection Questions

1. Why must SSH access be allowed before a default-deny firewall is enabled?

Because enabling default-deny before creating an SSH allow rule drops all inbound traffic — including the administrator's own SSH session. On a remote-only system this causes permanent lockout until console recovery. Allowing SSH first preserves administrative access while default-deny still applies to everything else.

2. What is the difference between an open port and an authorised service?

An open port is a socket in LISTEN state — the kernel accepts connections. An authorised service is one that a policy explicitly permits from defined sources. In this lab, port 8080 was open (Python HTTP server listening on 0.0.0.0:8080) but only authorised for source 192.168.56.20. From any other source, packets were dropped by UFW, making the port effectively closed despite the process still listening. Open ≠ reachable; reachable ≠ authorised.

3. Why does the server need the public key while the client must protect the private key?

The server stores the public key in `~/.ssh/authorized_keys` and uses it to verify cryptographic signatures the client produces during authentication. The client's private key signs the authentication challenge and is the sole proof of identity. Anyone with the private key can impersonate the user, so it must never leave the client. Public keys are designed for distribution; private keys are secrets.

4. Which rollback step would recover access after an invalid SSH configuration?

Restore the SSH config from the backup taken before changes, remove the invalid drop-in, validate, then reload — all through console access (not SSH):

text

```
sudo cp /etc/ssh/sshd_config.core102.bak /etc/ssh/sshd_config
sudo rm /etc/ssh/sshd_config.d/90-core102-hardening.conf
```

```
sudo sshd -t  
sudo systemctl reload ssh
```

In this lab's LXD adaptation, `lxc exec server-vm -- <command>` provides equivalent console access. The `with-ufw` LXD snapshot is the deeper fallback: `lxc restore server-vm with-ufw`.

Final Checklist

- [x] I worked only in an authorised laboratory environment.
- [x] I included my full name and registration number in the report.
- [x] Every required command and result is visible in the evidence.
- [x] I explained results instead of submitting screenshots alone.
- [x] I removed passwords, keys, tokens and unrelated personal information.
- [x] I opened the submitted files and confirmed that they are complete.